

IT UNIVERSITY OF COPENHAGEN

A Critical Evaluation of the LLM2Vec Method

A thesis submitted for the degree of

BSc Data Science

<https://github.com/Markererer/llm2vec>

Mark Assejev

Student 21390

BIBAPRO1PE

maass@itu.dk

Supervisor:

Rob van der Goot

IT University of Copenhagen

December 2025

Contents

1	Introduction	1
2	Background Work	3
2.1	Transformer Architectures	3
2.2	The LLM2Vec Method	3
2.3	MaChAmp and Gradual Unfreezing	4
2.4	Tasks and Datasets	5
3	Methodology	7
3.1	Experiment Design	7
3.2	Models	7
3.3	Training Strategies	8
3.4	Implementation and Infrastructure	9
3.5	Evaluation and Error Analysis	9
4	Results	10
4.1	Frozen Encoder Performance	10
4.2	Replication Validation	10
4.3	Frozen vs. Unfrozen Comparison	11
4.4	Summary	12
5	Analysis	13
5.1	Quantitative Analysis	13
5.2	General Observations	14
5.3	Qualitative Error Analysis	14
6	Discussion	17
6.1	Threats to Validity and Limitations	17
7	Conclusion	18
7.1	Summary	18
7.2	Addressing Research Questions	18
7.3	Future Work	19

Abstract

Decoder-only language models have recently been proposed as powerful text encoders when adapted through techniques such as LLM2Vec. Prior work has largely prioritized large models and lacks a systematic examination of how architectural choices, fine-tuning strategies, and dataset characteristics influence model performance on word-level tasks. This thesis presents a controlled empirical study evaluating Sheared LLaMA 1.3B as a decoder-only encoder on named-entity recognition (NER) and part-of-speech tagging (POS) benchmarks (CoNLL-2003 and UD English Web Treebank). I compare fully fine-tuned and frozen variants of the model across multiple training steps and benchmark against a strong encoder-only baseline, DeBERTa V3 Large. To explore an alternative, more stable optimization strategy, I utilize MaChAmp framework.

We observe that frozen decoder-only embeddings are as strong as in the original LLM2Vec research, achieving 96.6% accuracy on NER and outperforming full fine-tuning, which suffers from catastrophic forgetting and instability on small datasets. A hierarchy of difficulty is observed, with domain shift (EWT) substantially degrading unfrozen performance. Notably, while the decoder model performs well on standard benchmarks, the encoder-only baseline demonstrates superior robustness on noisy web text. Finally, it is concluded that gradual unfreezing with a dynamic learning rate training strategy effectively mitigates optimization instability, achieving a high combined XPOS/UPOS score of 1.96 on EWT. Overall, the study provides evidence that decoder-only models can serve as competitive text encoders under the right training regime and highlights the importance of layer-freezing strategies for a stable adaptation.

Introduction

In modern natural language processing (NLP), Large Language Models (LLMs) have become the dominant research focus [1], [2]. For token-level tasks, such as named-entity recognition (NER) and part-of-speech tagging (POS), encoder-only architectures, such as DeBERTa, are the main baselines [3]. Decoder-only architectures, like LLaMA [2], Mistral [4], and GPT-type models [1], demonstrate competitive performance across a range of tasks. Recent research suggests that through parameter-efficient modifications, such as LLM2Vec [5], decoder-only architectures can be transformed into powerful text encoders.[6]

The LLM2Vec approach involves enabling bidirectional attention on a causal attention model, retraining the model on masked next token prediction (MNTP), then running unsupervised contrastive learning[5]. It claims that after applying a sequence of these 3 steps, it can produce contextual embeddings that outperform popular encoder models on both word-level and sequence-level tasks [5]. While the original LLM2Vec paper evaluates multiple decoder-only architectures, its analysis of direct performance comparison of smaller models (such as Sheared LLaMA 1.3B) is incomplete [7]. Specifically, it lacks a systemic examination of how fine-tuning choices (frozen embeddings vs. full fine-tune vs. gradual unfreezing) influence word-level performance on different datasets.

While CoNLL-2003 remains a standard benchmark for NER [8], its usage for POS is considered outdated ever since the release of EWT (Universal Dependencies). CoNLL-2003 relies on the older Penn Treebank tag set [9], and is restricted to the news domain. In contrast, the UD English-EWT employs a modern universal POS (UPOS) standard, and is not limited to only news, but also blogs, reviews, social media posts, etc. This offers a more robust and linguistically representative representation of an encoder’s capabilities. [10]. This is why this study also utilizes the UD English-EWT dataset to align with the modern POS research practices.

This thesis conducts a controlled empirical study of LLM2Vec-style probing for word-level sequence labelling. I investigate the performance of Sheared LLaMA 1.3B under frozen and fully fine-tuned configurations on CoNLL-2003 (NER and POS) and English Web Treebank (EWT) tasks. To allow for a direct comparison with the original study [5], I also evaluate the encoder-only baseline, DeBERTa V3-large, using the same frozen probing setup. Since the original LLM2Vec implementation did not natively support DeBERTa for token-level tasks, I extended the codebase to enable these experiments.

To investigate whether structured optimization strategies can surpass the frozen probing baseline established by LLM2Vec, I apply gradual unfreezing to DeBERTa

V3 Large on the EWT tagging task. Gradual unfreezing allows deeper encoder layers to be incrementally released for training, reducing catastrophic forgetting and stabilising fine-tuning on a relatively small token-level dataset [11].

Following the original LLM2Vec work, I evaluate models primarily in terms of token-level accuracy. Although the extended implementation also tracks precision, recall, and F1-score during training and, for most configurations, at test time, accuracy is used as the main reported metric to maintain strict comparability with the published LLM2Vec results [5].

Research Questions

1. How do frozen and fully fine-tuned variants of a decoder-only model (Sheared LLaMA 1.3B) compare on NER, POS and EWT tagging tasks?
2. Does task difficulty (NER vs. CoNLL POS vs. EWT UPOS/XPOS) systematically influence the performance of decoder-only embeddings?
3. How do decoder-only embeddings from Sheared LLaMA 1.3B compare to encoder-only baselines based on DeBERTa V3 Large under the same probing setup?
4. Does gradual unfreezing improve stability and performance when fine-tuning DeBERTa V3 Large for EWT tagging?

Contributions

- A systematic comparison of frozen and unfrozen Sheared LLaMA 1.3B across CoNLL-2003 NER, CoNLL-2003 POS, and UD English-EWT tagging.
- An extension of the LLM2Vec codebase to support DeBERTa V3 Large as an encoder-only baseline within the same probing framework.
- A comparative evaluation of model performance across diverse domains, contrasting results on canonical news text (CoNLL) versus noisy web text (EWT).
- A practical evaluation of gradual unfreezing for DeBERTa V3 Large on EWT, examining whether structured partial unfreezing yields gains over purely frozen probing.

Background Work

2.1 Transformer Architectures

The Transformer architecture, introduced by Vaswani et al. [12], fundamentally changed natural language processing by dispensing with recurrence and convolutions in favor of a self-attention mechanism. This mechanism allows the model to weigh the significance of different words in an input sequence regardless of their distance, enabling highly parallelized training and the capture of long-range context dependencies.

Encoder-Only Architectures

Encoder-only models, such as BERT [13] and DeBERTa [3], utilize bidirectional attention. Bidirectional attention means that every token (j) in a sequence can attend to both previous and next tokens in the input ($1 \leq j \leq N$). This multi-token visibility makes it possible for the model to capture deep contextual information from both the left and right context simultaneously. Such an approach makes understanding the full context of the input sequence possible, which is crucial for resolving ambiguity. These architectures have been conventionally used as the standard baselines for token-level classification tasks (NER and POS tagging).

Decoder-Only Architectures

Decoder-only models, such as LLaMA [2], utilize causal (or masked) attention. This mechanism restricts how much the model can “see”: a token at position i can only attend to the previous tokens ($j \leq i$). This masks any future-positioned information, and “strictly prevents attending to future tokens, ensuring unidirectional information flow. For Autoregressive Generation, it is essential for next-token prediction, preserving the model’s generative nature. However, for Bidirectional Sequence Encoding, this constraint is detrimental, preventing the full leveraging of future context for rich, holistic token representations.” [14]

2.2 The LLM2Vec Method

The LLM2Vec framework transforms decoder-only language models into universal text encoders through a three-stage adaptation process. This methodology addresses the inherent limitations of causal attention mechanisms when applied to non-generative tasks.

1. Enabling Bidirectional Attention

The fundamental architectural change proposed by LLM2Vec is the removal of the causal attention mask and replacing it with the bidirectional attention mask. LLM2Vec replaces this constraint with an all-ones matrix, which converts the model into a bidirectional encoder. [5], [13]. However, naively enabling this access without further training often degrades representational quality, as the model’s weights were not optimized to process information from future tokens during its original pre-training phase [5].

2. Masked Next Token Prediction (MNTP)

In order to adapt the pre-trained model to its new bidirectional attention layer use, Masked Next Token Prediction (MNTP) training is used. [5].

Given an arbitrary sequence $x = (x_1, x_2, \dots, x_N)$ as input, we first mask a fraction of the input tokens and then train the model to predict the masked tokens based on the past and future context. Crucially, when predicting a masked token at position i , we compute the loss based on the logits obtained from the token representation at the previous position $i - 1$, not the masked position itself. [5]

This approach respects the original autoregressive nature of the decoder while forcing it to integrate bidirectional context.

3. Unsupervised Contrastive Learning

The first two steps adapt the model for token-level understanding, but they do not explicitly teach the model how to construct a unified representation for an entire sequence. LLM2Vec solves this by incorporating the SimCSE approach [15].

Given an input sentence, it is passed through the model twice with independently sampled dropout masks, resulting in two different representations for the same sentence. The model is trained to maximize the similarity between these two representations while minimizing the similarity with representations of other sentences in the batch. [5].

It is important to note for this thesis that while the third step refines global sequence embeddings, the original authors conclude that for word-level tasks (such as NER and POS tagging), applying the SimCSE step degrades the performance of the models. [5].

2.3 MaChAmp and Gradual Unfreezing

In order to compare the word-level tasks performance of the baseline model with an alternative training approach, this thesis utilizes the MaChAmp framework [11]. MaChAmp is a toolkit designed for easy fine-tuning of contextualized embeddings in multi-task settings.

A key feature of the framework utilized in this work is **gradual unfreezing**[16]. When applying a large pre-trained encoder to a specific downstream task, updating

all parameters simultaneously can lead to "catastrophic forgetting," which means that the model can overwrite its pre-trained knowledge before it can adapt to the new task. Gradual unfreezing mitigates this by initially freezing the entire encoder and training only the classification head. Over the following epochs, the framework incrementally unfreezes the encoder layers, starting from the highest layers (closest to the output) and moving down to the lowest layers. This strategy greatly stabilizes the optimization process and has been shown to yield better performance on smaller datasets [11], [16].

2.4 Tasks and Datasets

To evaluate the quality of the embeddings produced by the models, this study utilizes three distinct token-level classification tasks.

Named Entity Recognition (NER)

NER involves identifying and classifying proper nouns into predefined categories. We utilize the **CoNLL-2003** dataset [8], which annotates entities using 9 classes (e.g., PER, ORG, LOC).

Part-of-Speech Tagging (POS)

POS tagging assigns a grammatical category to each token. We evaluate on two datasets to contrast performance across different domains and granularities:

- **CoNLL-2003 POS:** Uses the standard Penn Treebank tag set (45+ tags). This is a fine-grained schema applied to canonical news text.
- **UD English EWT:** Represents the "noisy" web domain (blogs, reviews, social media) [10]. Crucially, we evaluate this dataset using two distinct label sets:
 - **UPOS (Universal):** Coarse-grained (17 tags) for cross-lingual consistency.
 - **XPOS (Language-specific):** Fine-grained (50 tags), utilizing the Penn Treebank scheme similar to CoNLL.

Table 2.1 provides a comparative example of these schemas.

Task	Dataset	Input Token	Gold Label
NER	CoNLL-2003	Apple Inc.	B-ORG I-ORG
POS (Fine)	CoNLL-2003	Dogs run	NNS (Noun, Plural) VBP (Verb, non-3rd pres.)
POS (Coarse)	UD EWT (UPOS)	Dogs run	NOUN VERB
POS (Fine)	UD EWT (XPOS)	Dogs run	NNS VBP

Table 2.1: Comparison of annotation styles. Note that CoNLL and EWT XPOS distinguish grammatical properties (like plurality and person), while UPOS generalizes them.

Methodology

3.1 Experiment Design

The experiments in this thesis follow the general probing philosophy of LLM2Vec: instead of fully fine-tuning large language models, I primarily treat them as frozen encoders and train small task-specific classifiers on top. The goal is to compare how different encoder architectures and training options support token-level tasks, and whether alternative fine-tuning setups can outperform the standard frozen embedding approach.

Concretely, I evaluate two pre-trained models on three token-level tasks:

- **Sheared LLaMA 1.3B**: A 1.3B parameter instruction-tuned decoder model [7].
- **DeBERTa V3 Large**: A strong encoder-only baseline [3].

The models are probed on the following datasets (detailed in Section 2.4):

- **CoNLL-2003 NER**: Evaluated on the standard test split.
- **CoNLL-2003 POS**: Evaluated using the fine-grained Penn Treebank tag set.
- **UD English-EWT**: Evaluated using both Universal (UPOS) and language-specific (XPOS) tag sets. For the primary frozen probing experiments, I report XPOS to ensure comparability with CoNLL.

For each model–task combination, I compare distinct training strategies. For **Sheared LLaMA 1.3B**, I evaluate both fully frozen and fully unfrozen configurations to assess the stability of the decoder-only architecture during fine-tuning. For **DeBERTa V3 Large**, I focus on the frozen baseline to maintain strict comparability with the original study, and additionally apply a **gradual unfreezing** strategy on the EWT dataset to investigate if structured optimization can yield further gains.

3.2 Models

Sheared LLaMA 1.3B

Sheared LLaMA 1.3B is used as a representative decoder-style large language model. Before I could run any experiments, I enabled bidirectional attention on the model, loaded from HuggingFace, and trained the model for 1000 steps on MNTP for further use. This adaptation was performed on the on English Wikipedia dataset [5],

which is a direct reproduction of the LLM2Vec approach. To ensure computational efficiency and stability, Low-Rank Adaptation (LoRA) is used during this phase. LoRA injects trainable rank decomposition matrices into the Transformer layers while keeping the pre-trained weights frozen. [17] This allows the model to adapt to the bidirectional objective without the instability associated with full-parameter updates. For all experiments, I load the base model and pre-trained (MNTP) checkpoint weights, then attach a token-level classification head that maps each token representation to the desired label set (NER, CoNLL POS, or EWT tags). For both frozen and unfrozen settings, the same hyperparameters as in the original paper [5] are used.

DeBERTa V3 Large

DeBERTa V3 Large is a strong encoder-only Transformer. I load it from HuggingFace and initialize it using the HuggingFace `AutoModelForTokenClassification` interface with the appropriate number of labels and label mappings for each dataset, the same as with the previous model. As with Sheared LLaMA, a task-specific classifier head is trained on top of the frozen encoder, and in some configurations, the encoder itself is gradually unfrozen.

3.3 Training Strategies

Frozen Encoders

In the frozen configuration, all encoder parameters are kept fixed, and only the task-specific classifier is trained. This matches the original LLM2Vec idea of probing representations without altering them. The classifier is a single linear layer on top of the final hidden state for each token, with output dimensionality equal to the number of labels in the task.

For each model and task, I train this classifier using cross-entropy loss, the AdamW optimizer, and a fixed learning rate of $5e - 4$. Training is performed for a predefined number of steps (e.g., 1500 and 3000) on the remote HPC cluster, with intermediate checkpoints saved for later evaluation. In the experiments, the next Nvidia GPUs are used: A100_40GB, V100, A30.

Unfrozen Encoders

In the unfrozen configuration, the entire model (encoder and classifier) is allowed to update during training. This setting is applied primarily to Sheared LLaMA 1.3B to study how sensitive a large decoder-style model is to full fine-tuning on relatively small token-level datasets. The same optimization setup as in the frozen case is used, but the number of trainable parameters is substantially higher (1,294,897,161), which makes the training much more sensitive to hyperparameters and training duration. Later, I evaluate the models on their 6,000th checkpoint.

Gradual Unfreezing for DeBERTa

For DeBERTa V3 Large on the EWT tagging task, I additionally explore a gradual unfreezing strategy [16], using MaChAmp [11]. MaChAmp employs a dynamic learning rate scheduler (slanted triangular learning rates). This scheduler linearly increases the learning rate during the initial training phase and then linearly decreases it, allowing the model to stabilize weights as the layers get progressively unfrozen, from the classifier to a fully-unfrozen model.[11] This schedule empirically leads to the best XPOS and UPOS scores reported in the results chapter.

Where relevant, I use a `retroactive_labels` setting such as `next_token` in the configuration files to control how labels are shifted relative to the input tokens, following the probing protocols of LLM2Vec.

3.4 Implementation and Infrastructure

The original LLM2Vec codebase does not natively support training DeBERTa V3 Large on the considered token-level tasks, so I extended the training and evaluation scripts to handle DeBERTa V3 Large via the HuggingFace `AutoModelForToken-Classification` interface while keeping the probing setup (frozen encoder with a small classifier head) compatible with the existing infrastructure. During training, models are periodically saved as HuggingFace checkpoints, and for encoder-based models, I additionally save the classifier head separately as `classifier.pt` so that the testing scripts (for example, the DeBERTa word-task test script) can load the base pre-trained model, replace its randomly initialised classifier with the saved `classifier.pt`, and run evaluation on the test set.

For improved reproducibility, I used the same fixed random seeds as the original authors did in their repository. [18].

It is important to note that, in some experiment configurations, I employed a different attention implementation from that used by the original LLM2Vec [5] authors due to hardware constraints. While they report using Flash Attention 2 [6] for all runs, certain configurations in this thesis were trained with the standard PyTorch [19] eager attention implementation [12]. This choice affects training throughput (speed) and memory usage, but maintains the mathematical equivalence to “flash attention 2” [6]. Ultimately, the difference in this implementation does not affect model weights or downstream task accuracy.

3.5 Evaluation and Error Analysis

Performance is compared across models (Sheared LLaMA, DeBERTa) and training strategies (frozen, unfrozen, gradual unfreezing). Checkpoints at different training steps (e.g., 1500, 3000, 6000) are evaluated to study how performance evolves over time.

Results

In this chapter, I report the quantitative results for the experimental setups described in Chapter 3. All scores represent token-level accuracies on the respective test sets. I first present the main results for the frozen configurations, followed by a validation of my replication against the original study. I then contrast these baselines with the fully unfrozen performance to highlight stability issues, and finally present the results of the gradual unfreezing strategy.

4.1 Frozen Encoder Performance

Table 4.1 presents the token-level accuracy for both Sheared LLaMA 1.3B and DeBERTa V3 Large in the frozen configuration (3000 steps), along with the original paper scores, which they report at 1500 steps [5]. This setup establishes the baseline performance for the models acting as pure feature extractors.

We observe that the decoder-only Sheared LLaMA 1.3B achieves **96.56%** on NER and **91.89%** on standard POS tagging. This is very close to the reported results in the original paper. While DeBERTa V3 Large (an encoder-only model) outperforms Sheared LLaMA on the complex EWT task (**94.42%** vs **91.62%**), the decoder model remains highly competitive on the simpler CoNLL tasks.

Model	NER	POS	EWT XPOS
Sheared LLaMA 1.3B	96.56	91.89	91.62
Sheared LLaMA 1.3B-llm2vec	96.59	92.04	-
DeBERTa V3 Large	91.26	73.72	94.42
DeBERTa V3 Large-llm2vec	94.97	86.49	-

Table 4.1: Token-level accuracy for frozen encoders at 3000 steps.

4.2 Replication Validation

To validate the local implementation of the LLM2Vec probing framework, Table 4.2 compares my replication results against the scores reported in the original paper [5].

My replication of Sheared LLaMA 1.3B at 3000 steps (**96.56%**) aligns closely with the reported score (**96.59%**), verifying the correctness of our bidirectional attention implementation. A discrepancy is observed for DeBERTa V3 Large (approx. 3.5%

difference), which may be attributed to hardware-specific attention implementations or undisclosed hyperparameter optimizations in the original study. The explanation on why both 1500 and 3000 steps are referenced here is in Subsection 6.1.1

Model	Original Paper Scores	1500 steps	3000 steps
Sheared LLaMA 1.3B	96.59	96.08	96.56
DeBERTa V3 Large	94.97	90.51	91.26

Table 4.2: Comparison of replicated results against original LLM2Vec reported scores for NER.

In this table, if we look at the values of Sheared Llama-1.3B, we can see that my runs done with 3000 steps are closer to the reported 1500 steps in the original paper[5]. A 3.5% accuracy difference is noticeable when the DeBERTa V3-Large reported score is compared to both of my scores.

4.3 Frozen vs. Unfrozen Comparison

Table 4.3 compares the frozen configuration (evaluated at 3000 steps) against the fully unfrozen strategy (evaluated at 6000 steps) for Sheared LLaMA 1.3B.

The results show a significant performance degradation in the unfrozen setting across all tasks. While the frozen model maintains high accuracy, the unfrozen variant drops by over 14% on CoNLL POS and over 46% on the EWT dataset. Such a big drop in performance can be explained with “catastrophic forgetting” (see section 2.3), which most likely results from too big of a learning rate.

Model	NER	POS	EWT_XPOS
Sheared LLaMA 1.3B (Unfrozen)	84.34	77.18	45.08
Sheared LLaMA 1.3B (Frozen)	96.56	91.89	91.62

Table 4.3: Performance comparison of Frozen vs. Fully Unfrozen strategies for Sheared LLaMA 1.3B. The unfrozen configuration suffers significant degradation across all tasks.

Gradual Unfreezing DeBERTa V3-large (EWT_POS)

Table 4.4 reports the effect of the gradual unfreezing schedule described in Section 3.3.3 on the EWT_POS tagging task.

The gradual unfreezing strategy on DeBERTa V3 Large for the EWT task shows consistent improvement over epochs.

- The model reaches a high combined score of 1.9649.
- Both XPOS and UPOS scores improve steadily, with the best model (Epoch 15) achieving 0.9810 and 0.9839, respectively.

Epoch/Steps	XPOS	UPOS	Combined Score
Epoch 4 (~ 1500)	0.9759	0.9797	1.9556
Epoch 7 (~ 3000)	0.9756	0.9809	1.9565
Epoch 14 (~ 6000)	0.9800	0.9833	1.9633
Best Model	0.9810	0.9839	1.9649

Table 4.4: Impact of Gradual Unfreezing on DeBERTa V3 Large for the EWT task. Scores indicate accuracy.

4.4 Summary

Overall, the results in this chapter provide the quantitative basis for the analysis in Chapter 5, where I relate the frozen and unfrozen configurations back to the probing objectives and error patterns.

Analysis

Throughout this chapter, I discuss model performance in terms of token-level accuracy to enable a direct and fair comparison to the published LLM2Vec results [5].

5.1 Quantitative Analysis

Sheared LLaMA 1.3B

Frozen vs. Unfrozen

Comparing the evaluation scores of the frozen and unfrozen embeddings configurations of Sheared LLaMa 1.3b shows major differences in performance across tasks.

- **NER Performance:** The frozen model achieves a significantly higher score (96.56) compared to the unfrozen model (84.34). This suggests that the pre-trained representations in the frozen model are already well-suited for NER, and full fine-tuning might be disrupting these features or overfitting.
- **Task Complexity:** Both models display that learning 9 labels (NER) is much easier than learning 50 labels (POS). This is evident through the score difference.
- **EWT Performance:** On UD_EWT POS tagging the frozen model performs slightly worse than on ConLL-2003 POS. This difference is attributed to the nature of the EWT dataset. The unfrozen models' accuracy suffers much more, dropping to 45.08.

Unfrozen Performance Issues

The reason for the low performance of the unfrozen model is most likely explained by suboptimal fine-tuning parameters. This shows that using hyperparameters optimal for a classifier head training is an ineffective choice for full fine-tune model training. It is worth noting that longer training (10,000 steps) only degraded model performance.

DeBERTa V3 Large

Frozen Performance and Domain Robustness

Contrary to the Sheared LLaMA model, which demonstrated consistent performance across tasks and datasets, DeBERTa V3 Large shows a distinct performance difference between the tasks on CoNLL-2003 and EWT XPOS tagging.

- **Robustness on EWT:** Despite the challenging architecture of UD_EWT dataset, DeBERTa achieves a higher token-level accuracy of **94.42%**, outperforming the frozen Sheared LLaMA 1.3B model (91.62%) by almost 3%. This might suggest that the natively bidirectional encoder architecture of DeBERTa is a better fit for a dataset with informal text than the adapted decoder model.
- **Sensitivity on CoNLL Tasks:** On the contrary, the standard CoNLL-2003 benchmark, performance is much lower than expected. The NER score (91.26%) is below the originally reported baseline score [5]. POS score sees an even bigger drop to 73.72%. As noted in the Discussion 6.1.2, this is likely attributable to implementation-specific factors in the code adaptation.

5.2 General Observations

- **Frozen vs. Unfrozen:** LLM2Vec is optimized to work with frozen embeddings exclusively, since only that configuration produces results with a reasonably high and stable performance, especially for Sheared LLaMa-1.3B, without requiring many steps (1,500 steps accuracy only improves by less than 0.5% at 3,000 steps). On the contrary, trying to get a good performance from fine-tuning a model using the LLM2Vec tool would most likely require a careful reconfiguration of the hyperparameters for the run.
- **Dataset Difficulty:** As evidenced by the lower scores in EWT_POS, EWT appears to be a more challenging dataset for Sheared LLaMa-1.3B. However, natively bidirectional encoder DeBERTa V3-Large outperforms Sheared LLaMA-1.3B, likely because it inherently supports bidirectional attention without requiring the adaptation layers necessary for the decoder model.

5.3 Qualitative Error Analysis

To understand the specific linguistic limitations of the frozen encoder on the Universal Dependencies English Web Treebank dataset, a normalized UPOS confusion matrix, created from DeBERTa V3-large EWT POS testing data, is examined (Figure 5.1). While the model achieves high overall accuracy, the confusion matrix reveals error patterns driven by the domain shift between pre-training data and the EWT test set.

Ambiguity in Nominal Categories

The most significant systematic errors occur within **SYM**, **PROPN**, and **ADJ**. The **SYM** tag is made up of symbols or emojis, as is most often misclassified by the model as

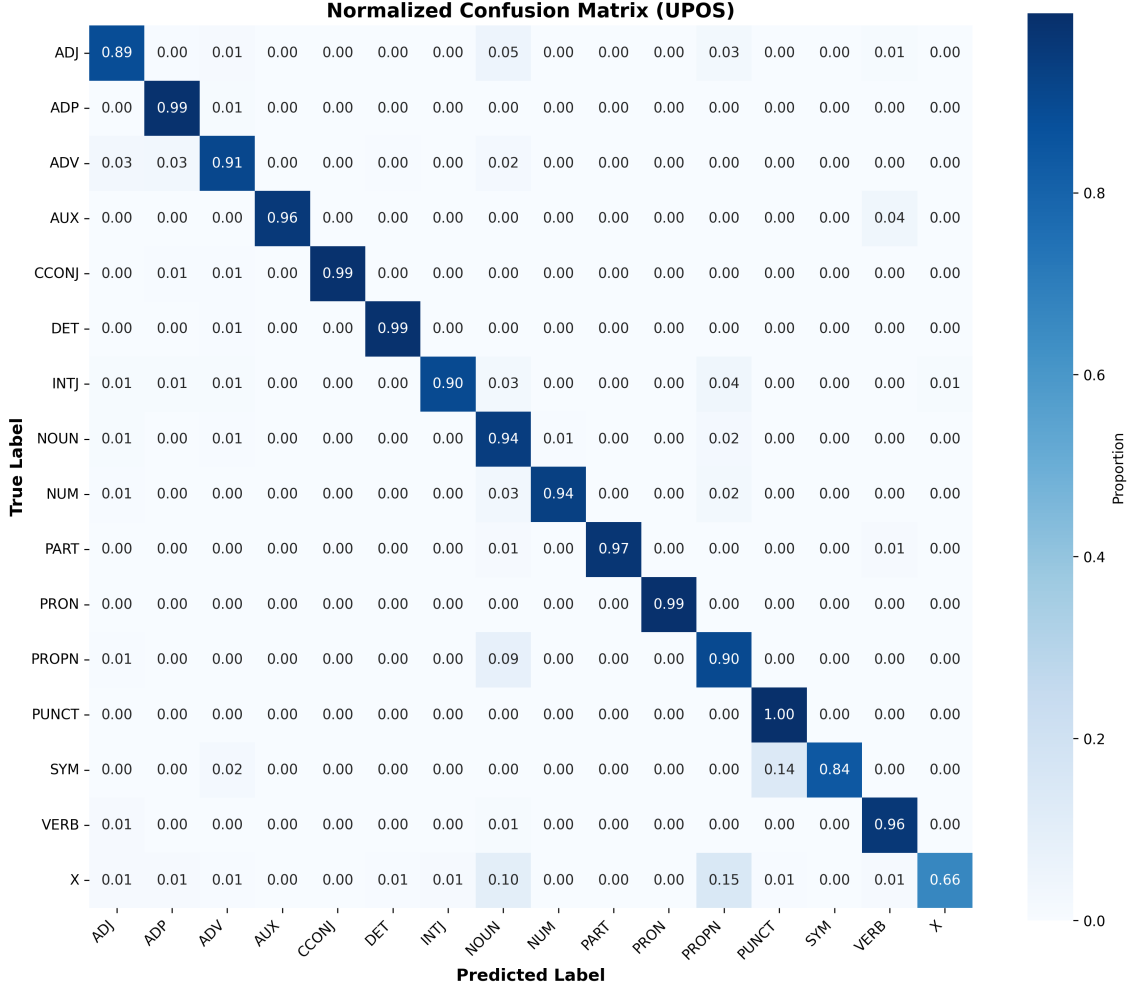


Figure 5.1: Normalized confusion matrix of UPOS predictions for the DeBERTa-v3-large (frozen) model evaluated on the EWT test set. Values represent per-class proportions, highlighting relative error patterns. The model achieves strong diagonal dominance across most categories, with remaining confusions concentrated mostly in X, SYM, and other nominal categories.

PUNCT. This can be expected, since emojis can also be expressed with punctuation and symbols (e.g., “.-.” or “,>”). [10] PROPN is often misclassified as NOUN, likely because of the dataset specifics. Since the EWT dataset contains social media text, which often lacks proper capitalization (e.g., “google” vs. “Google”), the model frequently defaults to predicting NOUN when such cues are missing. [10]

Finally, ADJ is predicted as NOUN likely because of cases where the adjective functions as the head of a nominal phrase (e.g., “the *poor*”). UD guidelines state that such instances have ADJ tag even though they could also function as a noun. [10]. Because of that, the model correctly identifies the nominal function, but incorrectly assigns the NOUN tag. This means that the model sometimes fails to adhere to the specific morphological priority of the UD schema.

Handling of Open-Class Items

The most confusion is within the X (Other) category, which includes gibberish text, emojis, word fragments, and unanalyzed foreign words. [10]. This class is most often misclassified as NOUN or PROPN. While the frozen embeddings are robust for standard English vocabulary, a distinct representation of "other" classes could not be captured within the model context. Therefore, the model tries to find the closest fit, which most often tends to be either of the 2 classes above.

Overall, the confusion matrix shows that the errors are concentrated in infrequent and open-class categories. It does not reflect major systematic syntactic weaknesses.

Discussion

While every effort was made to ensure fair, transparent, and reproducible evaluation, the following limitations should be considered when interpreting the results.

6.1 Threats to Validity and Limitations

This section highlights limitations that may have affected the reproducibility of the original paper [5] and the interpretation of the findings presented in this thesis.

Mismatch Between Reported and Actual Training Steps

While the original LLM2Vec paper states that word-level tasks were trained for 1,500 steps, in their released codebase on GitHub[18], configuration files consistently specify 3,000 training steps. This discrepancy complicates the comparison of the results. To address this mismatch, I evaluated the models on both 1,500 and 3,000 steps. The key finding is that the 3,000-step runs align more closely with the results reported in the paper. I emailed the authors to clarify this, but received no response. The result is that the original experimental setup [5] remains uncertain.

Implementation-Specific Factors

To improve reproducibility, all of the experiments are run using a fixed random seed. However, GPU-level nondeterminism may introduce minor variance. It is also important to point out that my own implementation for DeBERTa V3-large training and testing is very likely to differ from the original authors' code. However, since they did not publish the necessary code for this, the difference in evaluation scores of DeBERTa V3-large is expected. The same can be said about UD_EWT dataset implementation: in combination with my DeBERTa V3-large code adaptation, there might have been minor errors/bugs that were left unnoticed, directly impacting the evaluation scores.

Conclusion

7.1 Summary

This thesis investigated LLM2Vec-style probing for word-level sequence labelling with two pre-trained language models: the decoder-only Sheared LLaMA 1.3B and the encoder-only DeBERTa V3 Large. Building on the original LLM2Vec framework, I treated these models primarily as frozen encoders and trained lightweight token-classification heads for CoNLL-2003 NER, CoNLL-2003 POS, and UD English-EWT tagging.

The investigation yielded two primary insights. First, I verified that frozen decoder-only models can serve as powerful text encoders - the claim from the original paper [5]. Sheared LLaMA 1.3B achieved strong performance across all tasks in the frozen setting, successfully matching the paper-reported scores [5], but with training steps amount discrepancy 6.1.1. Second, full fine-tuning of that decoder model on small token-level datasets proved to be highly unstable, leading to catastrophic forgetting.

For the encoder-only baseline, extending the initially provided codebase [18] to support DeBERTa V3 Large led to a worse performance than in the original paper. While implementation specifics limited its performance on CoNLL NER and POS tasks, the model demonstrated superior robustness on noisy web text (UD_EWT), outperforming Sheared LLaMA. Additionally, gradual unfreezing - a completely different POS tagging training approach - proved to be effective for mitigating instability, yielding the highest reported scores for the POS task on the EWT dataset.

7.2 Addressing Research Questions

- **RQ1: How do frozen and fully fine-tuned variants of a decoder-only model (Sheared LLaMA 1.3B) compare on NER, POS, and EWT tagging tasks?**

The results conclusively show that frozen embeddings significantly outperform fully fine-tuned variants of the Sheared LLaMA 1.3B model on small token-level datasets. While the frozen model maintained high accuracy (e.g., 96.56% on NER), the fully unfrozen configurations suffered from severe optimization instability and catastrophic forgetting, with accuracy dropping by over 46% on complex tasks like EWT, likely due to suboptimal training hyperparameters.

- **RQ2: Does task difficulty (NER vs. CoNLL POS vs. EWT POS /**

XPOS) systematically influence performance?

Yes, it does - a clear hierarchy of difficulty was observed: **NER** (9 labels) proved to be the easiest, followed by **CoNLL POS** (45 tags), with **UD English-EWT** posing the biggest challenge due to its domain complexity. The performance gap between frozen and unfrozen models changed drastically as task difficulty increased, indicating that word-level tasks with a larger amount of labels make models more prone to optimization failure during fine-tuning.

- **RQ3: How do decoder-only embeddings from Sheared LLaMA 1.3B compare to encoder-only baselines based on DeBERTa V3 Large under the same probing setup?**

The comparison reveals that decoder-only models (Sheared LLaMA-1.3B) perform much better on word-level tasks using the CoNLL-2003 dataset, achieving scores significantly higher than the DeBERTa V3-Large implementation. However, the encoder-only baseline (DeBERTa V3-Large) showed better domain robustness, outperforming Sheared LLaMA-1.3B on the more diverse, informal text of the UD_EWT dataset (94.42% vs. 91.62%).

- **RQ4: Does gradual unfreezing improve stability and performance when fine-tuning DeBERTa V3 Large for EWT tagging?**

Yes. In contrast to the instability of full fine-tuning, gradual unfreezing, combined with a dynamic learning rate, improved the optimization process for DeBERTa V3 Large on the EWT dataset. This approach protected the model from forgetting pre-trained features and allowed it to adapt to the target domain, resulting in consistent epoch-over-epoch improvements and a final combined XPOS/UPOS score of 1.9649.

7.3 Future Work

Several directions for future research follow naturally from this work. The analysis could be extended beyond DeBERTa V3-Large and Sheared LLaMA 1.3B to also include other models from the original LLM2Vec paper, specifically other LLaMa variants and Mistral. This would provide more context on whether bigger decoder-only models show similar performance decrease on UD_EWT dataset. Additionally, more careful fine-tuning strategies can be tested, where the experiment setup would include comparing different learning rates. Third, while this thesis is only focused on token-level accuracy for the purpose of comparability with LLM2Vec, a more detailed evaluation report, which would include F1-scores, span-level metrics, could reveal more details of model behaviour. Finally, the analysis could be extended beyond English to include multilingual treebanks. This extension would test whether the observed patterns for Sheared LLaMA-1.3B and DeBERTa V3 Large exist across languages with different morphology and syntax.

Bibliography

- [1] T. B. Brown et al., *Language models are few-shot learners*, 2020. arXiv: 2005.14165 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2005.14165>.
- [2] H. Touvron et al., *Llama 2: Open foundation and fine-tuned chat models*, 2023. arXiv: 2307.09288 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2307.09288>.
- [3] P. He, J. Gao, and W. Chen, *Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing*, 2023. arXiv: 2111.09543 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2111.09543>.
- [4] A. Q. Jiang et al., *Mistral 7b*, 2023. arXiv: 2310.06825 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.06825>.
- [5] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, *Llm2vec: Large language models are secretly powerful text encoders*, 2024. arXiv: 2404.05961 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2404.05961>.
- [6] T. Dao, *Flashattention-2: Faster attention with better parallelism and work partitioning*, 2023. arXiv: 2307.08691 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2307.08691>.
- [7] M. Xia, T. Gao, Z. Zeng, and D. Chen, *Sheared llama: Accelerating language model pre-training via structured pruning*, 2024. arXiv: 2310.06694 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2310.06694>.
- [8] E. F. T. K. Sang and F. D. Meulder, *Introduction to the conll-2003 shared task: Language-independent named entity recognition*, 2003. arXiv: cs / 0306050 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/cs/0306050>.
- [9] M. P. Marcus, B. Santorini, and M. A. Marcinkiewicz, “Building a large annotated corpus of English: The Penn Treebank,” *Computational Linguistics*, vol. 19, no. 2, J. Hirschberg, Ed., pp. 313–330, 1993. [Online]. Available: <https://aclanthology.org/J93-2004/>.
- [10] N. Silveira et al., “A gold standard dependency corpus for english,” in *International Conference on Language Resources and Evaluation*, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:2339260>.
- [11] R. van der Goot, A. Üstün, A. Ramponi, I. Sharaf, and B. Plank, “Massive choice, ample tasks (MaChAmp): A toolkit for multi-task learning in NLP,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, D. Gkatzia and D. Seddah, Eds., Online: Association for Computational Linguistics, Apr. 2021, pp. 176–197. DOI: 10.18653/v1/2021.eacl-demos.22. [Online]. Available: <https://aclanthology.org/2021.eacl-demos.22/>.

- [12] A. Vaswani et al., *Attention is all you need*, 2023. arXiv: 1706.03762 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1706.03762>.
- [13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. [Online]. Available: <https://aclanthology.org/N19-1423/>.
- [14] Gemini, 2025. [Online]. Available: <https://gemini.google.com/share/f6c24166dbec>.
- [15] T. Gao, X. Yao, and D. Chen, “SimCSE: Simple contrastive learning of sentence embeddings,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds., Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 6894–6910. DOI: 10.18653/v1/2021.emnlp-main.552. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.552/>.
- [16] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, I. Gurevych and Y. Miyao, Eds., Melbourne, Australia: Association for Computational Linguistics, Jul. 2018, pp. 328–339. DOI: 10.18653/v1/P18-1031. [Online]. Available: <https://aclanthology.org/P18-1031/>.
- [17] E. J. Hu et al., *Lora: Low-rank adaptation of large language models*, 2021. arXiv: 2106.09685 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2106.09685>.
- [18] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy. [Online]. Available: <https://github.com/McGill-NLP/llm2vec>.
- [19] A. Paszke et al., *Pytorch: An imperative style, high-performance deep learning library*, 2019. arXiv: 1912.01703 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1912.01703>.